Lecture 1

January 8, 2026 3:41 PM

Unit 1 - Summarizing Data – Graphical Methods

Introduction – Descriptive versus Inferential Statistics

What is/are statistics?

Statistics is the branch of mathematics concerned with making sense of data: collecting, summarizing, organizing, presenting, and analyzing numerical information.

Descriptive statistics covers methods for organizing and summarizing possibly rather large collections of data.

Example – GitHub repository statistics dataset (built-in or simulated dataset).

Suppose we collect data on 100 GitHub repositories:

- Number of stars
- Number of contributors
- Lines of code
- Number of issues
- Programming language

100×5 = Dimension of Data

We may want to summarize this data using representative numerical values:

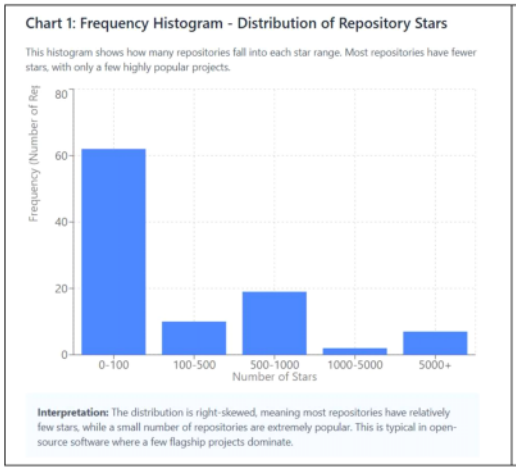
- Minimum stars: 5
- Maximum stars: 15,234
- Mean stars: 892

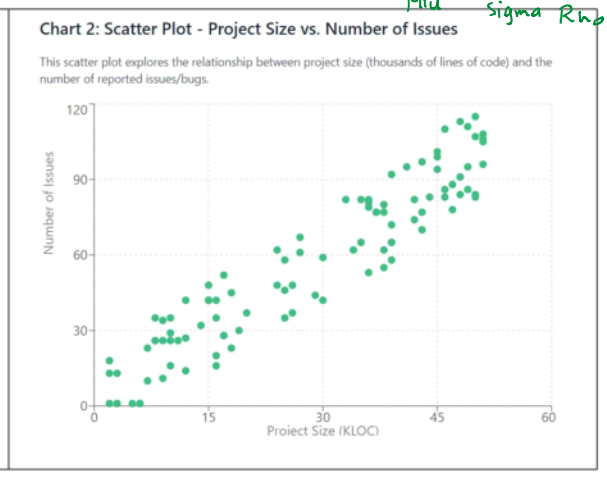
Sample (statistics : $\bar{x}$, s, r, ...)

Population (Parameters : μ, σ, ρ ....)

Miu sigma Rho

We can also visualize data in various ways:





This is descriptive statistics, and such charts and values usually describe a *sample* taken from a larger *population*.

Inferential statistics (or statistical inference) provides methods for (1) estimating values of population parameters (statistical estimation) or (2) evaluating the degree of support for a

statement or claim about some characteristic of a population (hypothesis testing) using data obtained from a random sample of that population.

Examples:

1. What proportion of open-source projects use continuous integration?
2. Projects with more contributors have fewer bugs per 1000 lines of code.

Descriptive Statistics

Two main approaches:

- ✓ Graphical Methods
- ✓ Numerical Methods (Numerical measurements of position and variation)

Graphical ways of summarizing data: selecting the chart or graph that best visualized the data, depends on a number of parameters such as data type, type of relation or attribute that needs to be explored, and the number of data points. Here are some examples

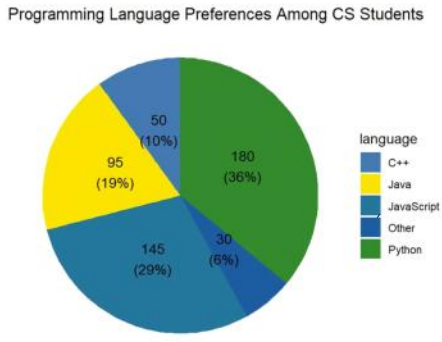
- ✓ Pie Chart: used only for categorical data to explore part-whole relationships
- ✓ Bar Charts: used for categorical data to compare frequencies or counts across categories
- ✓ Frequency Histograms: used for numeric data to explore the shape of the distribution
- ✓ Scatter Plots: used for two-attribute numeric data to explore correlations and trends
- ✓ Box Plots: used for numeric data to explore distribution characteristics (spread, central tendency, quartiles) and to compare distributions across multiple groups. Outliers are detected.

Categorical (Qualitative) Data: Pie Chart and Bar Chart

Pie charts are generally a poor way to represent quantitative data. However, they provide a simple way to visualize qualitative data.

Example: Suppose we survey 500 computer science students about their preferred programming language for personal projects:

- Python: 180 students ÷500 x 360
- JavaScript: 145 students ÷500 x 360
- Java: 95 students ÷500 x 360
- C++: 50 students ÷500 x 360
- Other: 30 students ÷500 x 360



We can represent this information in a pie chart to show the proportion of students preferring each language.

Bar Plots for Categorical Data

While pie charts provide a simple way to visualize categorical data, **bar plots** (or bar charts) are often more effective for comparing categories, especially when there are many categories or when precise comparisons are needed.

What is a Bar Plot?

A bar plot displays categorical data with rectangular bars. The height (or length) of each bar represents the frequency or count of each category. Unlike histograms, the bars in a bar plot are typically separated by gaps to emphasize that the data is categorical (discrete categories) rather than continuous.

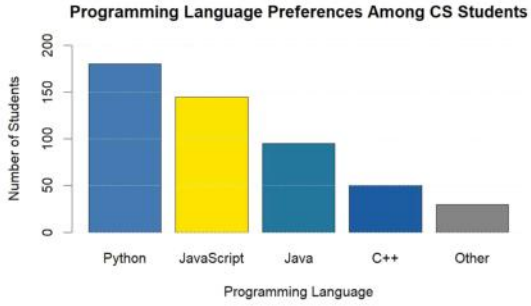
Example: Programming Languages Used in Projects

Suppose we survey 500 computer science students about their preferred programming language for personal projects:

Language	*Frequency*
Python	180
JavaScript	145
Java	95
C++	50
Other	30

We can represent this data using a bar plot where:

- The x-axis shows the programming languages (categories)
- The y-axis shows the number of students (frequency)
- Each bar's height represents how many students prefer that language



Advantages of bar plots over pie charts:

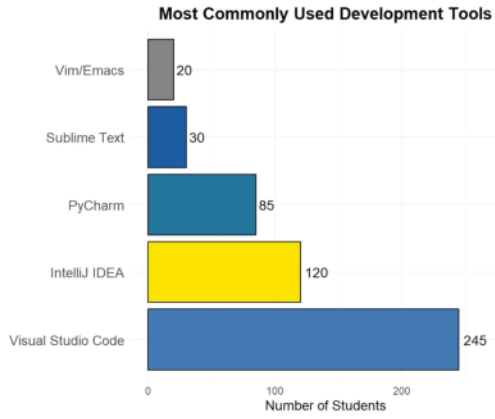
- ✓ Easier to compare values precisely
- ✓ Works well with many categories
- ✓ Can be oriented horizontally for long category names
- ✓ Can show multiple groups side-by-side (grouped bar plots)

Horizontal Bar Plots

When category names are long, we can create horizontal bar plots where categories are on the y-axis and frequencies are on the x-axis. This is particularly useful for displaying software framework names, library names, or tool names that might be lengthy.

Example: Most commonly used development tools:

- Visual Studio Code: 245 students
- IntelliJ IDEA: 120 students
- PyCharm: 85 students
- Sublime Text: 30 students
- Vim/Emacs: 20 students



When to use a Bar Plot:

- You have categorical data
- You want to compare frequencies or counts across categories
- You need precise comparisons between categories
- You have multiple groups to compare (grouped bar plots)

Box Plots (Box-and-Whisker Plots)

A **box plot** is a standardized way of displaying the distribution of data based on five key summary statistics:

1. Minimum
2. First Quartile (Q1) - 25th percentile
3. Median (Q2) - 50th percentile
4. Third Quartile (Q3) - 75th percentile
5. Maximum

No outliers

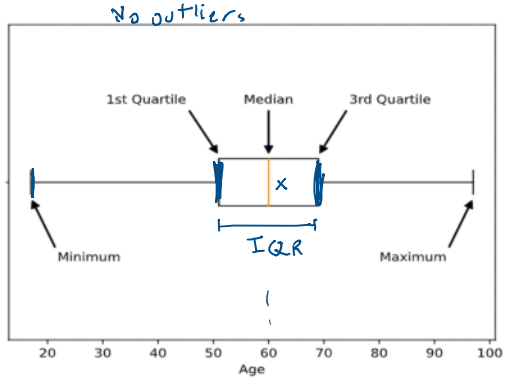


Image Source: https://www.w3schools.com/statistics/sta 1

$IQR = Q_3 - Q_1$

Anatomy of a Box Plot

- **Box**: Represents the middle 50% of the data (from Q1 to Q3), called the Interquartile Range (IQR)
- **Line inside box**: The median value
- **Whiskers**: Extend to show the range of the data (typically to minimum and maximum, or to 1.5×IQR)
- **Outliers**: Points that fall outside the whiskers (shown as individual dots)

MIN ↑ $Q_1 = 25^{th}$ Med ↑ 50th ↑ Q_2 ↑ $Q_3 = 75^{th}$ MAX

Example: API Response Times

Example: Suppose we measure API response times (in milliseconds) for 100 requests:

Summary statistics:

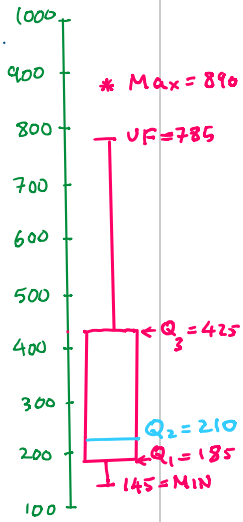
- Minimum: 145 ms
- Q1: 185 ms
- Median: 210 ms
- Q3: 425 ms
- Maximum: 890 ms

IQR = 425 − 185 = 240 ms

LF = 185 − 1.5 × 240 = −175 ms → N/A

UF = 425 + 1.5 × 240 = 785

There are upper outliers.



Detecting Outliers with Box Plots

Box plots are excellent for identifying outliers. The standard definition of an outlier is:

- Any value below: Q1 - 1.5 × IQR Lower fence
- Any value above: Q3 + 1.5 × IQR Upper fence

Where $\boxed{\text{IQR = Q3 - Q1}}$

In our API example: IQR = 425 - 185 = 240 ms, Lower fence = 185 - 1.5(240) = -175 ms (no values below this)

Upper fence = 425 + 1.5(240) = 785 ms

Any response times above 785ms would be shown as outlier points. These might represent requests that hit uncached data, database timeouts, or network issues.

What can we learn from this box plot?

- The median response time is 210ms
- The middle 50% of responses are between 185ms and 425ms
- ✓ The data is right-skewed (upper whisker and upper half of box are longer) = Positive skew (long tail on the larger numbers)
- The large spread between Q3 and maximum suggests some requests are much slower

When to use a boxplot:

- You have continuous numerical data
- You want to show the distribution, spread, and central tendency
- You need to identify outliers
- You want to compare distributions across multiple groups

Side-by-Side Box Plots

One of the most powerful features of box plots is the ability to compare distributions across different groups using **side-by-side box plots** (also called **comparative box plots**).

Example: Comparing Algorithm Performance

Suppose we want to compare the execution time (in milliseconds) of three different sorting algorithms on the same dataset:

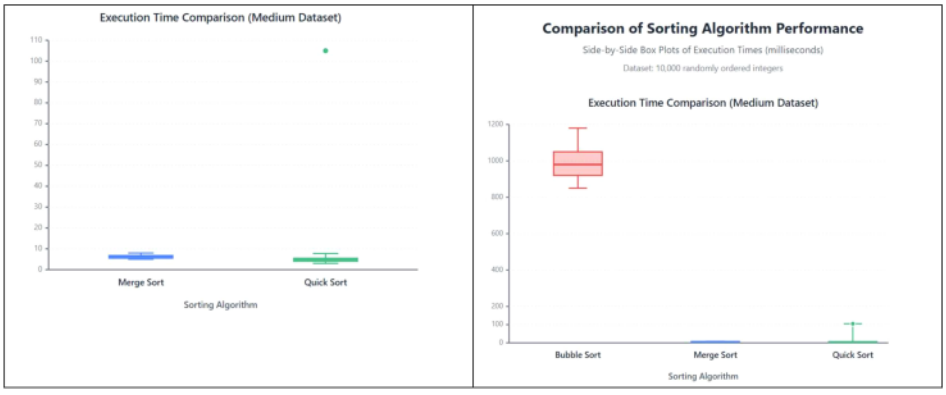
Algorithms:

1. Bubble Sort
2. Merge Sort
3. Quick Sort

Suppose we run each algorithm 50 times and record the execution time. Instead of looking at three separate box plots, we place them side-by-side on the same graph.

What we might observe:

Statistic	Bubble Sort	Merge Sort	Quick Sort
Minimum	850 ms	5 ms	3 ms
Q1	920 ms	5.5 ms	4 ms
Median	980 ms	6.2 ms	4.8 ms
Q3	1050 ms	6.8 ms	5.5 ms
Maximum	1180 ms	8 ms	105 ms



Insights from side-by-side box plots:

- **Quick Sort** has the lowest median execution time (~50ms)
- **Merge Sort** has consistent performance (small IQR, few outliers)
- **Bubble Sort** is dramatically slower (median ~1355ms) and all three distributions don't overlap
- **Quick Sort** has more variability and a few outliers (worst-case scenarios)

Quantitative Data

We will be mostly focusing on quantitative data in this course.

When we have a raw data set, it is often difficult to get information from it, especially if the data set is large. Consider, for example, a dataset of response times (in milliseconds) from a web API over 272 requests.

Example response times: 156, 167, 203, 178, 445, 421, 198, 402, ...

It is hard to obtain any meaningful information from this dataset in its current form. How can we organize the data?

Suppose we are mostly interested in roughly how long the API responses took.

Stem-and-Leaf Plot

A stem-leaf plot allows us to group the data without losing any information. To create one, we divide the data into stems – based on the leftmost significant figures in the data – and the leaves.

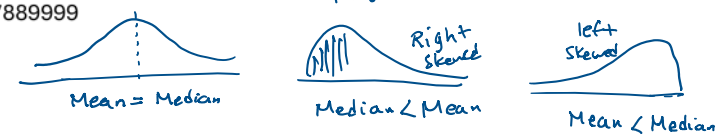
Here is an example stem-leaf plot for API response times (in milliseconds):

Key: 1 |5 = 150 (The decimal point is 2 digit(s) to the right of the |)

Right skewed Distribution

1 | 5678889999
2 | 0001112233445567788899
3 | 0233456788
4 | 0001122334455667889999
5 | 0112234
6 | 23
7 | 8
8 | 5
9 | 12
10 | 7

Eeach data point is represented in this plot.



For instance, the first row represents response times: 150ms, 160ms, 170ms, 180ms, 180ms, 180ms, 190ms, 190ms, 190ms, 190ms.

What can we say about the response times?

- Most responses are clustered in two groups: 150-300ms (fast responses) and 400-500ms (slower responses)
- There are some outlier responses taking 700-1000+ ms
- This bimodal distribution might indicate cached vs. uncached requests

---

Frequency Distribution and Histogram

For large data sets, a stem and leaf plot can be unwieldy and gives us more information than we need. Instead, we can form intervals (classes) and tally the number of values within each interval.

Rules:

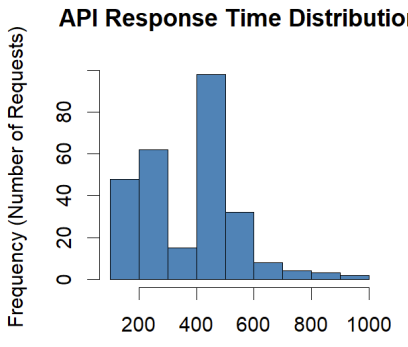
- The classes should all be the same width
- The classes should capture all the data
- The classes should not overlap

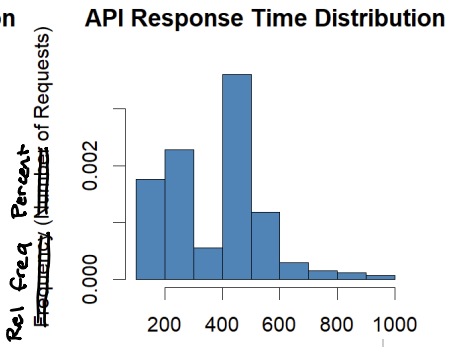
Here's a frequency table for API response times (in milliseconds):

$$k = 1 + \log_2 n = 1 + \frac{\log \overset{272}{n}}{\log 2} \approx 9$$

Response Time (ms)	Frequency = count
[100,200)	48
[200,300)	62
[300,400)	15
[400,500)	98
[500,600)	32
[600,700)	8
[700,800)	4
[800,900)	3
[900,1000)	2

$\sum f_i = 272$





$$\frac{\text{Range} = \text{Max} - \text{Min}}{9 = \text{number of classes}}$$

For example, there were 48 requests out of 272 that had response times between 100ms and 200ms. Note the brackets: [100,200) denotes a class that includes 100ms responses but excludes 200ms ones. This is so that there is no overlap between classes.

It may still be hard to make sense of this data in the above form, so we can represent it graphically with a histogram.

A **histogram** visualizes the frequency distribution. The height of each bar represents the frequency of values in that interval.

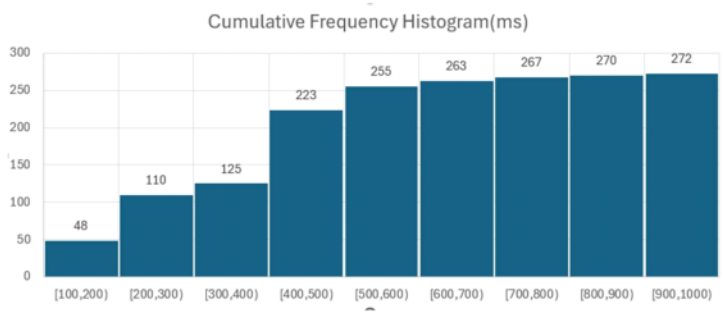
We want a number of classes so that we can obtain meaningful information about the data at a glance. Between 5 and 20 classes is a standard guideline, with the classes chosen so that the numbers are "nice" (e.g., a class of [100,200) is nicer than a class of [103,197)).

Cumulative Frequency Table

Often we are interested in knowing how much of our data is less than a certain value. For instance, we might be interested in knowing how many API requests had response times less than 500ms (which might be our SLA target).

Using intervals of 100ms, here's a cumulative frequency table:

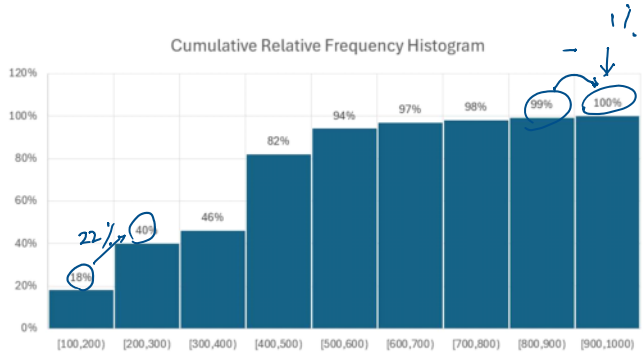
Response Time (ms)	Cumulative Frequency
[100,200)	48
[200,300)	110
[300,400)	125
[400,500)	223
[500,600)	255
[600,700)	263
[700,800)	267
[800,900)	270
[900,1000)	272



This table tells us that **223** of the requests had response times less than 500ms. Note that the cumulative frequencies increase.

By dividing each cumulative frequency by the total number of requests (272), we obtain a **cumulative relative frequency table**:

Response Time (ms)	Cumulative Relative Frequency
[100,200)	0.18
[200,300)	0.40
[300,400)	0.46
[400,500)	0.82
[500,600)	0.94
[600,700)	0.97
[700,800)	0.98
[800,900)	0.99
[900,1000)	1.00



Here we can see that **82%** (or 0.82) of requests had response times less than 500ms.

Use a Histogram when:

- You have continuous numerical data
- You want to see the shape of the distribution
- You're interested in frequency within ranges/bins